

Class #8

Architecture-Centric Design Method

Software Architectures · Master in Informatics Engineering

Cláudio Teixeira · claudio@ua.pt

Agenda

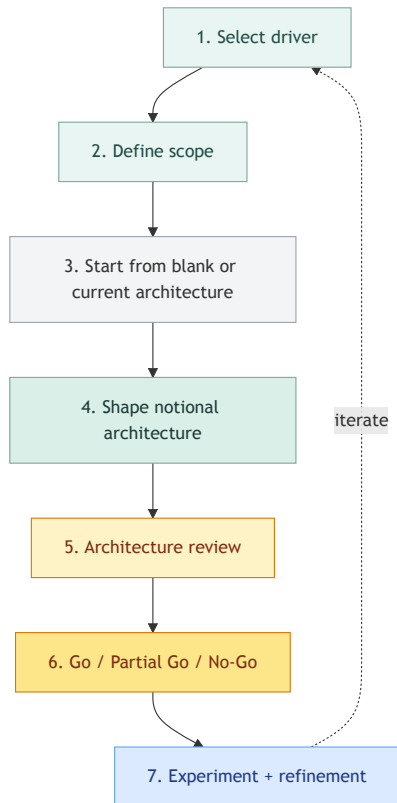
- **Block 1 – Architecture-Centric Design Method (ACDM)** 35 min
 - driver selection, scope, and current proposal
 - review board, go/no-go, and experiment charter
- **Block 2 – ACDM Walkthrough: HPMS** 45 min
 - proposal packet
 - readiness call
 - experiment and refinement
- **Block 3 – Group Exercise: RocketTickets Using ACDM** 100 min
 - build a review packet on a familiar case
 - respond to review findings
 - make the call and refine

Block 1

Architecture-Centric Design Method (ACDM)

ACDM at a glance

Architecture-Centric Design Method is a top-down design method that selects one architectural driver, scopes it, tests an architectural response, and loops until the proposal is strong enough to proceed.



Discover architectural drivers

Step 1 starts with stakeholder conversations, not with a diagram.

What you gather	Why it matters
Design objectives	clarify what the architecture is trying to optimize
Primary functional requirements	anchor the pass in real system behavior
Quality attribute scenarios	name the pressures that shape structure
Constraints and concerns	make limits and risks explicit early

Then prioritize the quality attribute scenarios so one pass has one dominant architectural driver.

Pick one driver for this pass

An ACDM pass becomes weak when it tries to solve the whole catalogue at once.

Weak framing	Better framing
"Make the system scalable, secure, usable, and integrated."	"Protect traceability of sensitive updates."
"Improve the architecture."	"Isolate an unstable external dependency from the core workflow."

The selected driver decides what the review should challenge. If another concern also matters, keep it as a guardrail for the pass, not as a second selected driver.

Establish project scope

Step 2 reviews and cleans up the inputs from Step 1 before design starts.

Step 2 activity	Output
Consolidate overlaps	remove duplicate drivers and repeated requirements
Complete unclear information	turn vague concerns into measurable or testable scenarios
Set boundaries	define what this pass includes and excludes
Refine the project frame	revised requirements, quality metrics, timelines, and development boundaries

Choose the starting point for the notional architecture

ACDM can start from either of these:

Starting point	When it fits
Blank interaction / blank architecture sketch	early design, no credible current structure yet
Existing interaction / current architecture	brownfield evolution, where the system already behaves in production

For real systems, the second option is often stronger because it makes change explicit. The goal of this step is to establish the baseline architecture that the pass will now reshape.

Create and cross-check the notional architecture

Once the driver and scope are chosen, ask:

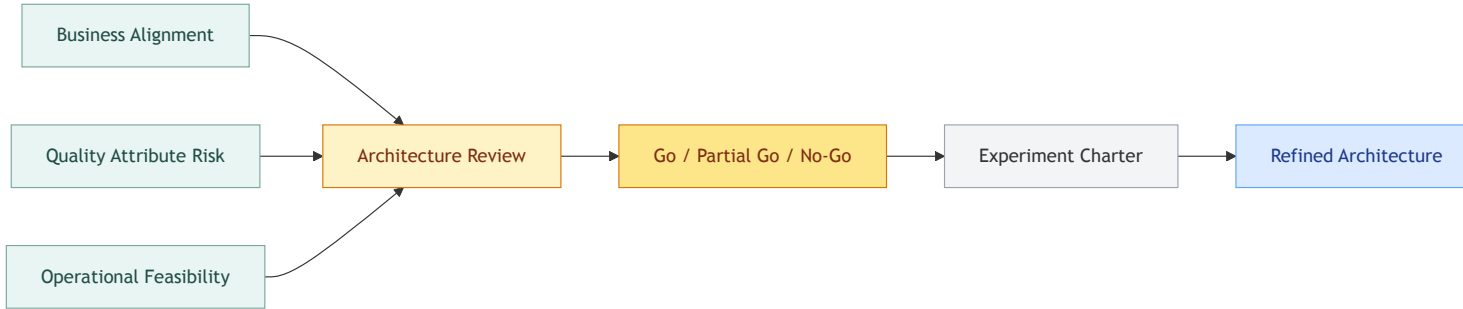
- which boundary becomes too wide
- which dependency is too close to the critical path
- which responsibility must become explicit
- which interaction should become asynchronous, isolated, or observable

The notional architecture should now make visible:

- modules, interfaces, components, and interactions
- key cross-cutting concerns
- the baseline structure to review with stakeholders
- enough structure to cross-check against the driver, scope, constraints, and principles

The point is not to redraw everything. It is to show how this driver reshapes the current architecture, and to leave behind something concrete enough for stakeholder review and iteration.

Architecture review and decision flow



What review and go/no-go are for

Architectural review is not a presentation ritual. It exists to surface:

- business misalignment
- unresolved quality attribute risk
- operational or delivery feasibility problems
- assumptions still too weak to trust

Alternative tactics, patterns, and trade-offs should be discussed explicitly here so the team can judge whether the best credible option has been chosen for the pass.

Go means "credible enough to proceed." **Partial go** means "some parts can proceed, others still need proof." **No-go** means "the current proposal is still too risky."

What makes a good experiment and refinement step

Experiments should target the thing most likely to invalidate the proposal.

Good examples:

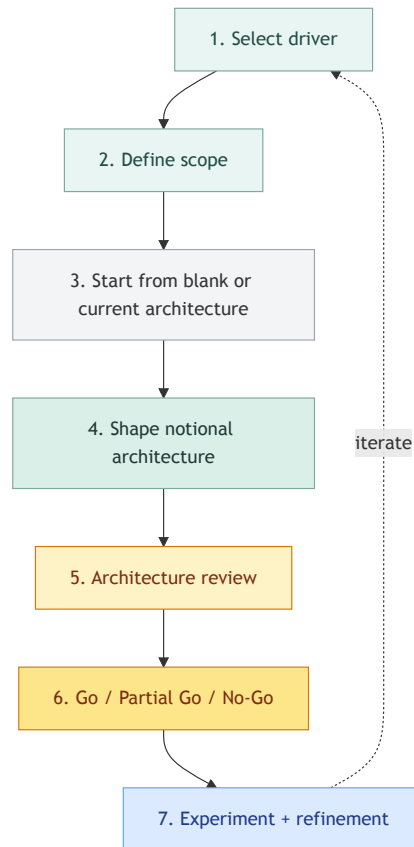
- load spike against the hottest workflow
- failure injection on a critical dependency
- security validation for a sensitive access boundary
- prototype of an unstable integration edge

Experiments may validate a `go`, strengthen a `partial go`, or challenge an unresolved risk. Record the results, refine the architecture if needed, and then loop back into review. If the result would not change the architecture, it was probably the wrong experiment.

What a solid ACDM pass leaves behind

A strong ACDM-based checkpoint should show:

1. explicit drivers
2. credible scope
3. a notional architecture with ownership and data boundaries
4. review findings, not just the polished diagram
5. a justified go/no-go stance
6. one concrete experiment or spike
7. a refined architecture that changed for a reason



Block 2

ACDM Walkthrough: HPMS

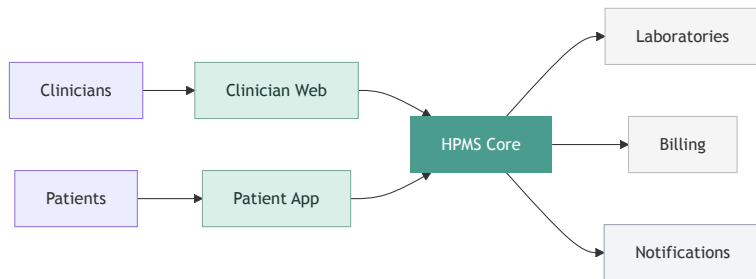
HPMS – system context

Healthcare Patient Management System:

- clinicians manage appointments and update clinical records
- patients use web or mobile channels for their side of the journey
- notifications keep both sides informed
- laboratories and billing remain external systems

What makes this architecturally interesting:

- clinical data is sensitive and must be traceable
- active care workflows should not stall because one dependency is slow
- support and compliance need to explain what changed, when, and why



HPMS – Step 1 input snapshot

1. SELECT DRIVER

These are the kinds of inputs ACDM would gather from stakeholder conversations before selecting a driver.

Input type	HPMS example
Functional requirements	create and update patient records, manage appointments, track treatments
Quality attribute pressures	protect sensitive data, keep workflows usable, support growing usage
Constraints	integrate existing healthcare systems, support web and mobile access, deliver under a fixed timeline
Principles	interoperability, modularity, continuous delivery

HPMS – from context to candidate drivers

1. SELECT DRIVER

Given that context, several plausible drivers emerge for one ACDM pass:

Candidate driver	Why it is plausible
Privacy / auditability	sensitive writes must be reconstructable
Responsiveness	care workflows cannot feel slow
Reliability under dependency failure	lab and billing systems are unstable edges
Operational supportability	support and compliance need explainable system state

HPMS – selected driver: what it means

1. SELECT DRIVER

SELECTED DRIVER

DRIVER STATEMENT

Clinical-record auditability

Protect the system's ability to reconstruct and defend every sensitive clinical-record change.

WHAT THAT MEANS HERE

- every sensitive record change should be traceable
- the system should show who changed what, when, and under which authorization
- the trace should survive failures and be credible during review or investigation
- audit evidence should not depend on clinicians manually reconstructing the story afterward

HPMS – Step 2: refine and scope the pass

2. DEFINE SCOPE

Before shaping the architecture, Step 2 cleans up and narrows the Step 1 inputs.

Step 2 decision	Result for this pass
Consolidate overlapping concerns	treat privacy + traceability as one auditability-focused driver
Keep one secondary pressure as a guardrail	lab dependency failure should not stall active care workflows
Make the main concern testable	a sensitive clinical-record change should be reconstructable end to end
Set boundaries	focus on record updates, access control, audit path, and the lab edge

HPMS – selected driver and scope

1. SELECT DRIVER • 2. DEFINE SCOPE

SELECTED DRIVER

Protect clinical-record auditability.

SCOPED TO

- clinician record updates
- access control
- audit trail
- lab integration edge

OUT OF SCOPE

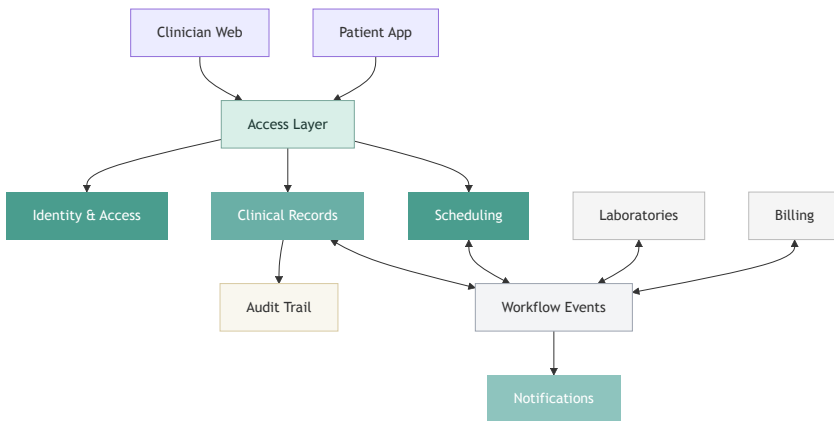
- billing optimization
- analytics platform
- patient-facing recommendations

GUARDRAIL FOR THIS PASS

- lab dependency failure should not stall active care workflows

STARTING POINT FOR THIS PASS

The baseline architecture for this pass is the current proposal under review.



HPMS – starting point for this pass

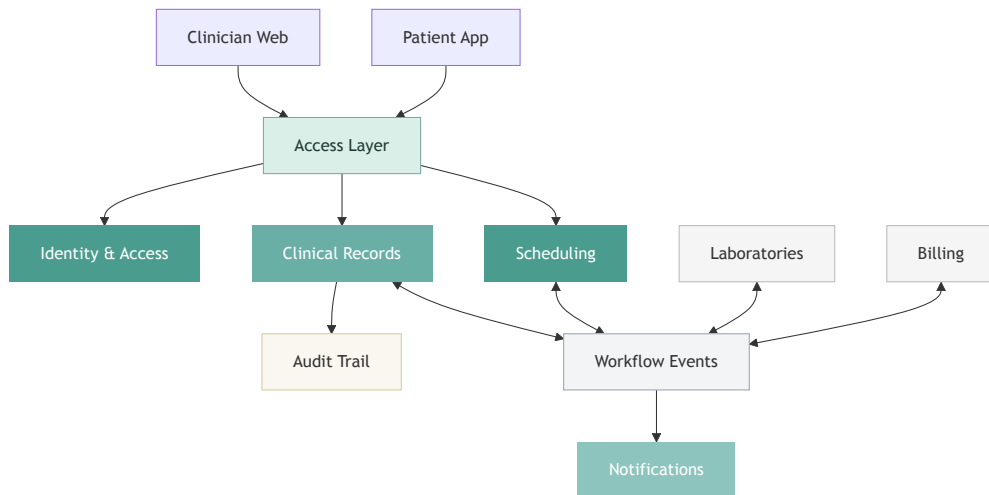
3. START FROM BLANK OR CURRENT ARCHITECTURE

SELECTED DRIVER

Protect clinical-record auditability.

Guardrail: lab dependency failure should not stall active care workflows.

We start from the current architecture rather than a blank sketch.



- this is the current interaction under review for the pass
- it already names the main modules, interfaces, and interactions
- it is the baseline architecture that Step 4 will now challenge

HPMS – shaping the notional architecture

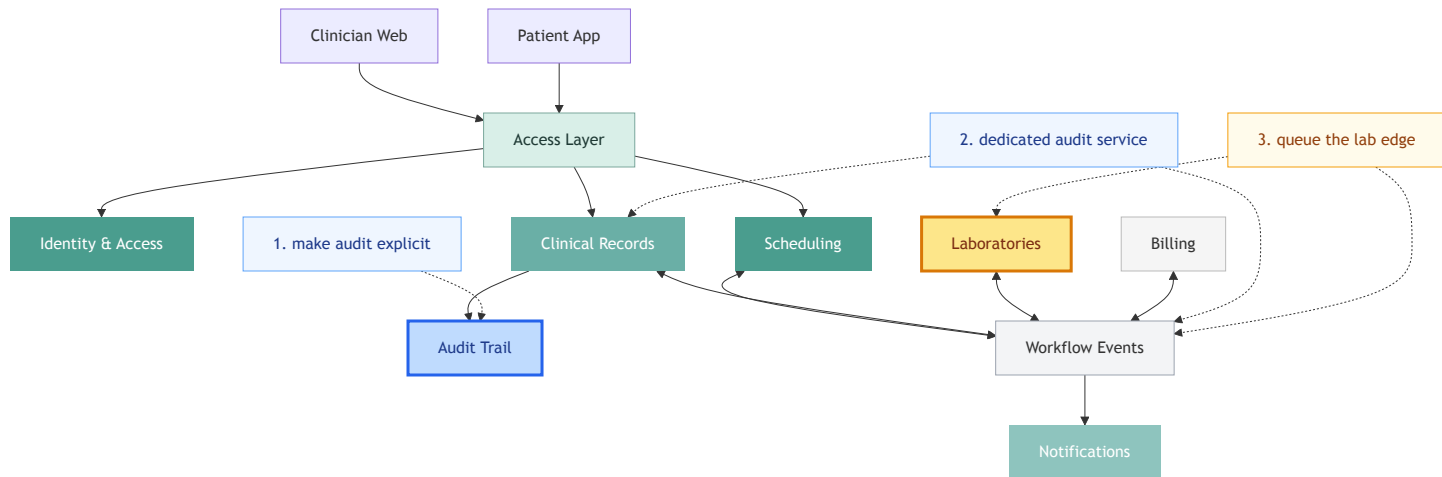
4. SHAPE NOTIONAL ARCHITECTURE

SELECTED DRIVER

Protect clinical-record auditability.

Guardrail: lab dependency failure should not stall active care workflows.

Before review, shape the baseline into a notional architecture by making a few credible moves that respond to the driver and the guardrail.



Option 1: keep audit close to Clinical Records, but make the audit write explicit and trustworthy.

Option 2: move toward a dedicated audit service connected to the workflow backbone.

Option 3: isolate Laboratories behind an asynchronous edge so failures do not leak inward.

These are candidate tactics and patterns, not yet approved decisions.

HPMS Review Board

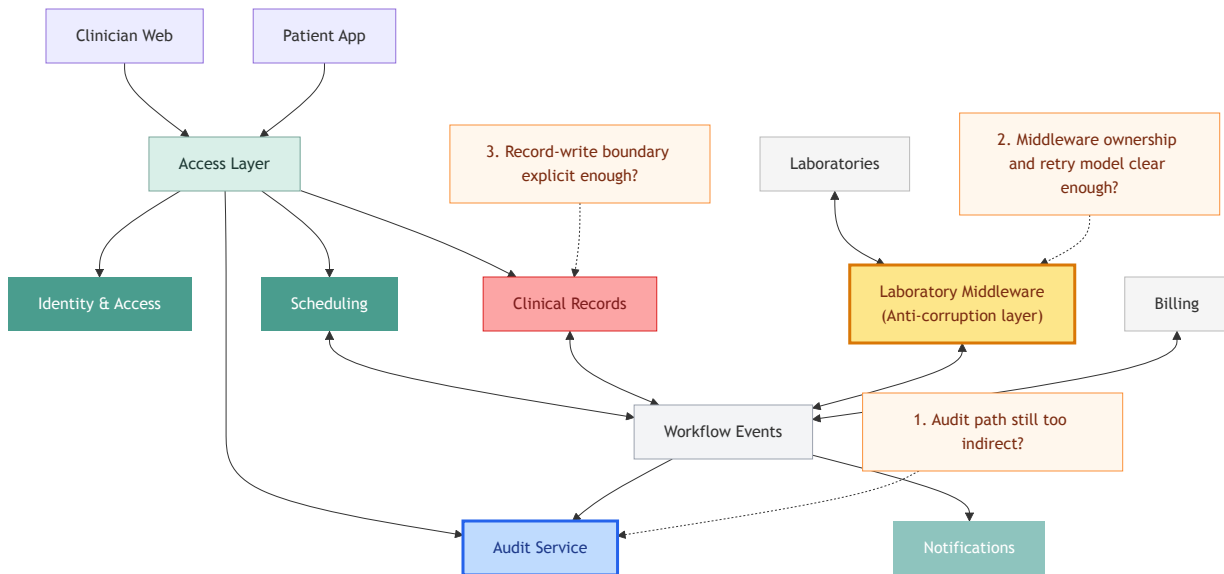
5. ARCHITECTURE REVIEW

SELECTED DRIVER

Protect clinical-record auditability.

Guardrail: lab dependency failure should not stall active care workflows.

Given the selected driver and this revised notional architecture, what concerns still remain before we can trust the chosen tactics?



HPMS – the readiness call

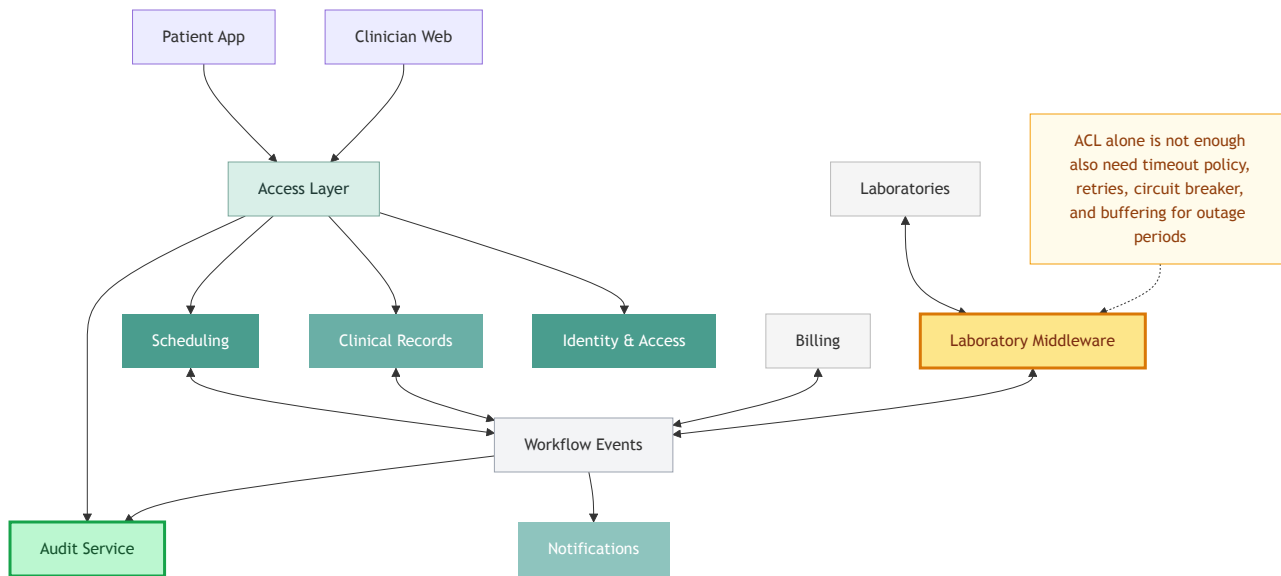
6. GO / PARTIAL GO / NO-GO

SELECTED DRIVER

Protect clinical-record auditability.

Guardrail: lab dependency failure should not stall active care workflows.

Turn the review findings into decisions about what can proceed now, what can proceed with caution, and what still needs more evidence.



Go in green: **Audit Service** as the main response to the selected driver

Partial go in yellow: **Laboratory Middleware** as the guardrail response for lab isolation

Resolved in this pass: **Clinical Records** now stays neutral

the output is a decision about what moves into refinement

HPMS – experiment charter

7. EXPERIMENT + REFINEMENT

SELECTED DRIVER

Protect clinical-record auditability.

Guardrail: lab dependency failure should not stall active care workflows.

Design only the experiments that could change the decision above. They can validate a current `go` , strengthen a `partial go` , or challenge an unresolved concern.

Experiment	Success signal
Audit write-path spike	a sensitive record update leaves complete, trustworthy audit evidence end to end
Lab outage and retry simulation	lab instability stays isolated and core workflows remain usable during intermittent failure
Event backlog simulation	audit and lab-related events remain explainable and recoverable under delayed delivery

These are the experiments worth doing because their results could still change the architecture.

HPMS – architecture after refinement

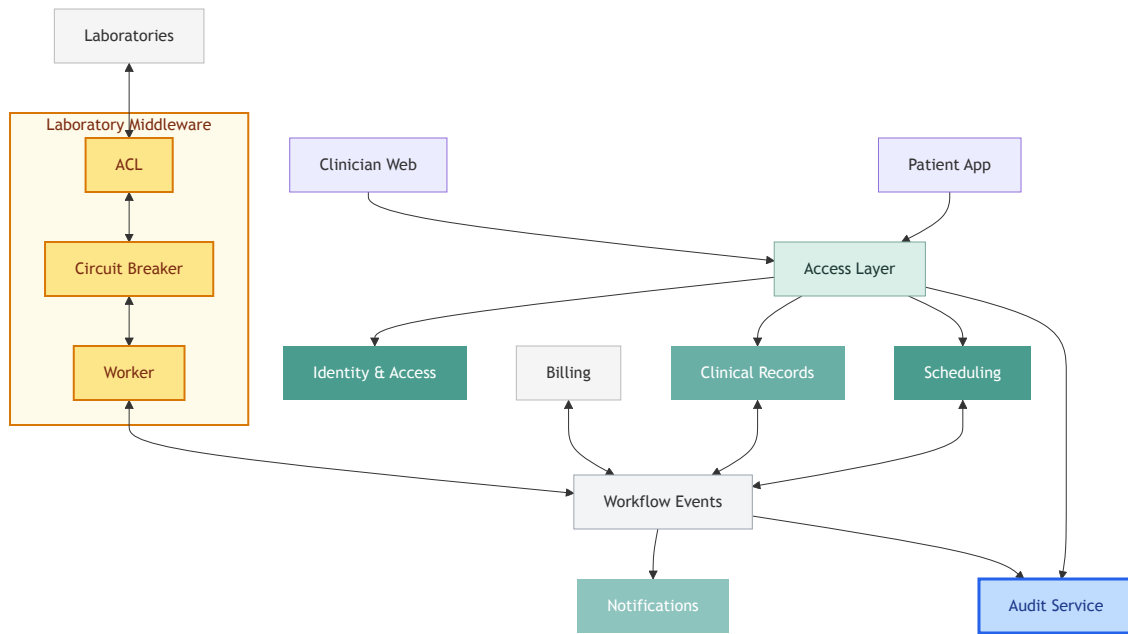
7. EXPERIMENT + REFINEMENT

SELECTED DRIVER

Protect clinical-record auditability.

Guardrail: lab dependency failure should not stall active care workflows.

This is the refined architecture after the chosen experiments have improved the design.



Audit Service now carries the explicit audit response

Laboratory Middleware now bundles the guardrail tactics into one boundary
the pass ends with a refined architecture, not just a review note
if later experiments change this proposal, the loop goes back to review

Block 3

Group Exercise: RocketTickets Using ACDM

Exercise

RocketTickets: trusted validation across channels

Use the same RocketTickets domain from the previous class.

RocketTickets sells through direct channels, promoter-operated channels, authorized resellers, attendee transfers, and digital wallets. Every ticket must still be trusted at the gate after resale, revocation, partner delay, or partial connectivity.



Exercise handout →

[docs.google.com/document/d/
16-roUQjFN6imwX0l9WzPPzDKzC5VIt95g1qVoSMRyFU](https://docs.google.com/document/d/16-roUQjFN6imwX0l9WzPPzDKzC5VIt95g1qVoSMRyFU)

35'

PART 1

4

PER GROUP

Part 1 Output

Build the review packet in 35'

Choose the drivers

pick one dominant driver from the candidate set

add one supporting driver if it really matters

Define the scope

what part of RocketTickets this pass covers

what stays outside this pass

Choose the starting point

use the first-cut starting architecture from the handout

treat it as the current proposal under review

Show the current proposal

annotate, redraw, or extend that baseline

make ownership history and credential-regeneration decisions explicit

List open review questions

2 to 4 things you are not yet comfortable approving

35'

GROUP WORK

4

PER GROUP

Review Injection

New information from the review board

Assume the review board raises these concerns:

Customer support **MUST** prove whether a transferred ticket was still owned by the original buyer when a PDF was regenerated.

Fraud investigations **MUST** be able to reconstruct a clear chain from issuance to transfer to reissue to revocation.

Exercise Part 2

Make the Call in 25'

Using the review findings, produce:

Go / partial go / no-go

what can proceed

what cannot yet be approved

Structural consequence

what should now become explicit in the architecture

why it must surface

One experiment charter

what you will test

what result would change the architecture

One refinement

redraw one boundary, subsystem, or interaction that changed because of the review

25'

REFINEMENT

20'

SHARE-OUT + DEBRIEF

Debrief lens

When we review outputs, we will look for:

- whether the selected driver is narrow enough for one ACDM pass
- whether the review findings actually changed the design
- whether the go/no-go decision is honest and scoped
- whether the chosen experiment attacks the riskiest assumption
- whether the refinement is traceable to review evidence

The best answer is not the most detailed architecture. It is the one that changed for defensible reasons.



Revisit – Home Reading

RocketTickets Worked Proposal

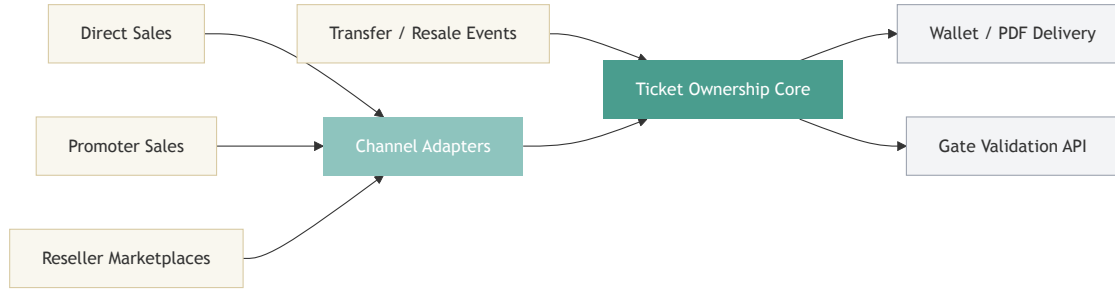
This is a compact solution proposal prepared for the Lecture 8 exercise handout. Use it as one possible answer, not as the only credible design.

Proposal packet – selected driver and scope

Step	Example output
Dominant driver	trusted gate validation despite delayed partner updates
Supporting concerns	auditability of ownership changes, partner autonomy, partial connectivity
Scope	partner ingestion, ownership authority, gate-facing trust state
Out of scope	pricing, marketing, seat allocation, venue staffing

This is narrow enough for one ACDM pass. It is not the whole RocketTickets platform.

Proposal packet – current proposal



This is a credible current proposal, but it still hides timing and trust problems.

Review injection – sample findings

Finding	What it pressures
Gate devices should not depend on a live central round-trip	operational feasibility at venue entry
Partner revocations can arrive late	freshness of trusted validation state
Ownership disputes need a clear lineage	auditability and authority
Delivery can regenerate credentials while ownership is still changing	separation of delivery from ticket truth
Batch-only promoters need tolerated lag and reconciliation rules	partner boundary and sync strategy

One credible readiness call

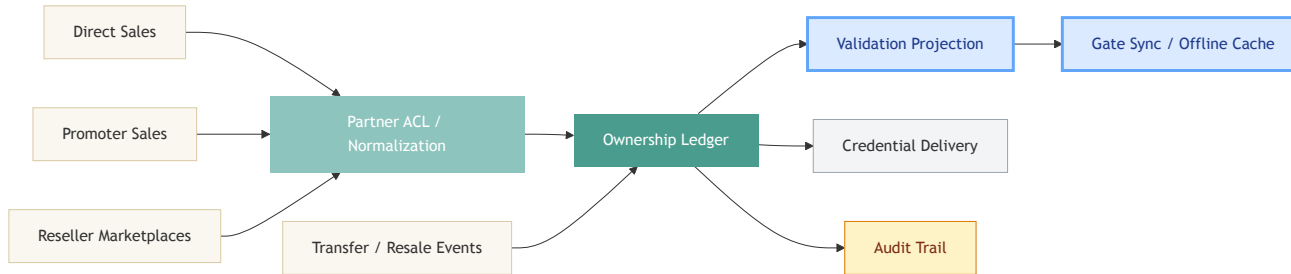
- **Go** on partner adapters and a central ownership authority
- **Partial go** on delivery, but only if it stops owning ticket truth
- **No-go yet** on direct gate queries to the ownership core
- **Required experiment** on revocation lag, offline validation, and reconciliation timing

The point is not to punish the architecture. The point is to decide what can responsibly move forward.

Example experiment charter

Experiment	Success signal	If it fails...
revocation lag simulation	gate-facing state stays trustworthy despite delayed partner updates	validation needs stricter projection and freshness rules
offline gate validation drill	entry devices answer quickly without calling the ownership core	add an explicit gate sync / cache boundary
ownership dispute trace	every transfer and revoke can be reconstructed end-to-end	audit lineage must become an explicit architectural responsibility

Refined architecture after review



Notice what changed:

- trusted gate state became a separate projection
- venue operation got its own sync / offline boundary
- delivery stopped owning ticket truth
- audit lineage stopped being an implicit promise

References

Bibliography

- Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Ford, N., Richards, M., Sadalage, P., Dehghani, Z. *Software Architecture: The Hard Parts*. O'Reilly Media.
- Richards, M., Ford, N. *Fundamentals of Software Architecture*. O'Reilly Media.
- Ingeno, J. *Software Architect's Handbook*. Packt.

Design Patterns & More

- Reference material PDF: [design_pattern_cheatsheet_v1.pdf](#)
- Medium cheat sheet: [medium.com/cp-massive-programming/design-patterns-cheat-sheet-list-e9a55d82de5d](#)
- GoF book: [learning.oreilly.com/library/view/design-patterns-elements/0201633612/](#)
- Software Architecture: The Hard Parts: [learning.oreilly.com/library/view/software-architecture-the/9781492086888](#)

Further Reading

- O'Reilly course playlist: [learning.oreilly.com/playlists/e4bbd2ea-5604-419c-8f40-2b0a06e7f8aa](#)